附录

附录 A 任务三：单机环境实现与功能验证

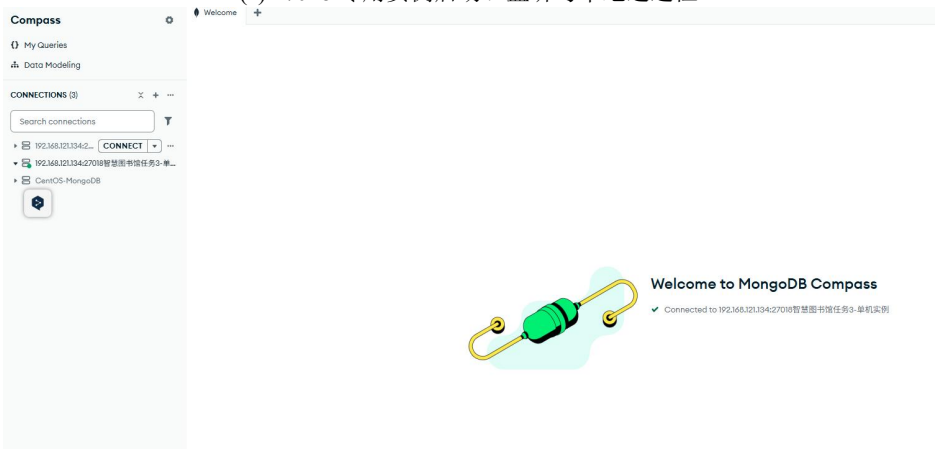
本附录对应正文第 4 章的单机数据库实验部分，依次记录专用实例部署、基础数据导入、连接故障排查、馆藏检索与借阅统计、索引优化及用户权限验证。各截图保留实验过程及结果说明，作为正文相关结论的补充证据。

A.1 单机环境与数据初始化

本节先确认独立实例能够正常启动并被客户端访问，再核对基础集合与数据规模；若远程连接失败，则通过防火墙规则排查并恢复访问。

```
[root@nosql01 ~]# runuser -u mongod -- mongod --config /etc/mongod-smart-library.conf
about to fork child process, waiting until server is ready for connections.
forked process: 2141
child process started successfully, parent exiting
[root@nosql01 ~]# ss -lntp | grep 27018
LISTEN 0      4096         127.0.0.1:27018      0.0.0.0:*    users:(("mongod",pid=2141,fd=10))
LISTEN 0      4096   192.168.121.134:27018      0.0.0.0:*    users:(("mongod",pid=2141,fd=11))
[root@nosql01 ~]# mongosh --host 127.0.0.1 --port 27018 --quiet --eval 'db.runCommand({ping:1})'
{ ok: 1 }
```

(a) 27018 专用实例启动、监听与本地连通性



(b) Windows Compass 成功连接专用实例

图 A-1 专用 MongoDB 实例与客户端连接

本图由服务器端启动 MongoDB 8.0.32 专用实例后，依次检查 27018 端口监听状态，并在 Windows 端使用 Compass 建立连接所得。上下两部分分别对应服务端运行状态和客户端连接结果，用于证明单机实验环境已经具备本地运行与远程图形化访问条件。

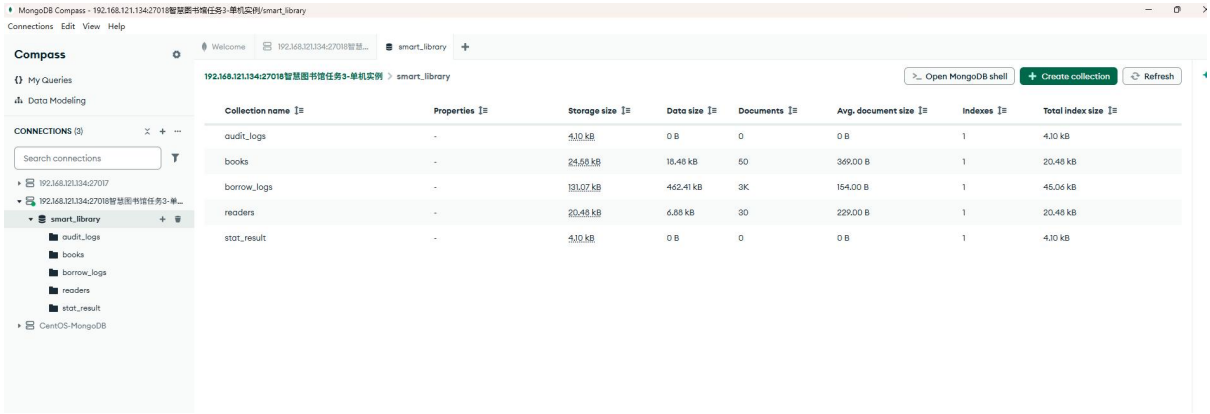
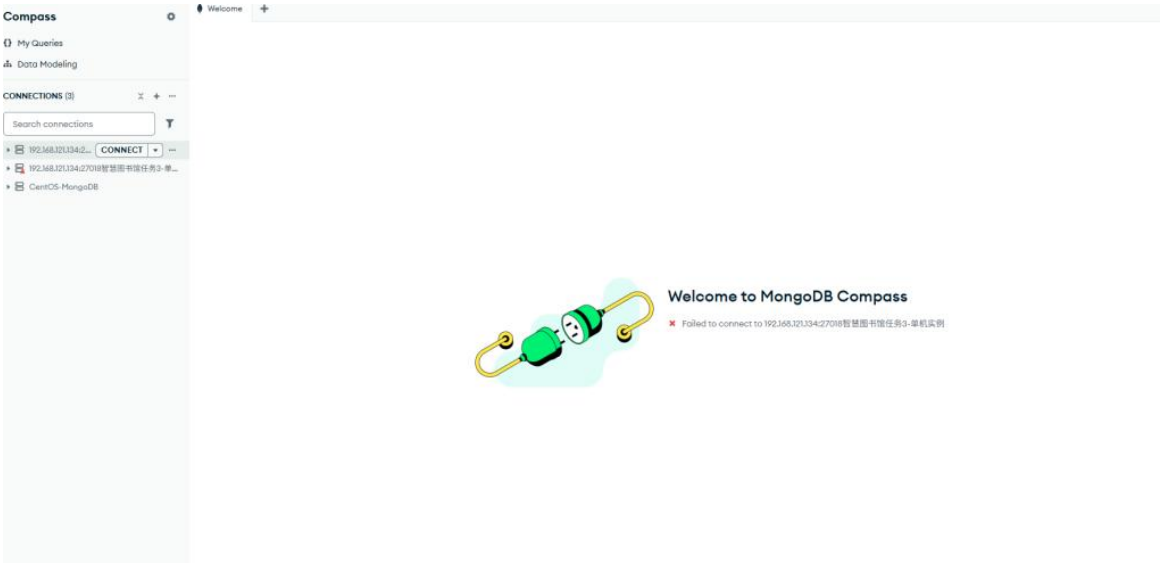


图 A-2 初始化阶段集合与基础数据数量

该截图来自 Compass 对 smart_library 数据库集合及文档数量的检查。通过核对 books、readers 和 borrow_logs 等集合，确认任务数据已导入，基础数据规模与任务要求相符，为后续检索、统计、索引和应用功能测试提供统一的数据基线。



(a) Windows Compass 首次连接 27018 失败

```
[root@nosql01 ~]# firewall-cmd --state
running
[root@nosql01 ~]# firewall-cmd --list-ports
27017/tcp
[root@nosql01 ~]# firewall-cmd --get-active-zones
public (default)
  interfaces: ens33
[root@nosql01 ~]# firewall-cmd --permanent --add-port=27018/tcp
success
[root@nosql01 ~]# firewall-cmd --reload
success
[root@nosql01 ~]# firewall-cmd --list-ports
27017/tcp 27018/tcp
```

(b) firewalld 永久放行 27018/tcp

图 A-3 远程访问故障定位与处理

本图记录了 Windows 端首次连接 27018 失败后的排查过程：先保留 Compass 连接失败现象，再在服务器端检查防火墙规则并永久放行 27018/tcp，随后重新加载规则。该证据用于说明远程连接问题并非数据库实例未启动，而是主机防火墙未开放对应端口，处理后客户端才具备访问条件。

A.2 检索 统计与索引

在数据基线确认后，依次运行馆藏检索和聚合统计脚本，并通过 explain() 检查高频查询的执行计划，形成从功能正确性到查询效率的验证链条。

```
[通过] 三个集合的数据量均与源文件一致。
(base) PS E:\Projects\smart-library-project> .\.venv\Scripts\python.exe scripts\task3_queries.py

任务三：6 类检索执行结果

[Q1] 计算机类 + 价格 50-100 元
结果条数：8
------------------------------------------------------------
[Q2] 同时包含"编程"和"入门"标签
结果条数：1
------------------------------------------------------------
[Q3] 书名包含"Python"
结果条数：2
------------------------------------------------------------
[Q4] 出版年份为 2021-2023
结果条数：5
------------------------------------------------------------
[Q5] 缺少 tags 字段
结果条数：4
------------------------------------------------------------
[Q6] 评分降序 Top 10
结果条数：10
------------------------------------------------------------
books 集合总数：50

[通过] 6 类检索均已执行完成。

执行记录已保存：E:\Projects\smart-library-project\docs\任务3_检索执行记录.md
(base) PS E:\Projects\smart-library-project>
```

(a) 六类检索执行汇总



(b) 五项统计执行汇总

图 A-4 检索与统计功能运行结果

图中上半部分为六类馆藏检索脚本的执行汇总，下半部分为五项聚合统计任务的运行结果。结果由 Python 调用 MongoDB 查询语句和 aggregate 聚合管道生成，分别覆盖条件筛选、标签匹配、模糊检索、日期范围、字段存在性、排序分页，以及分类统计、学院借阅量、年月趋势和热门图书等业务，用于核对功能输出与预期结果是否一致。



图 A-5 索引清单与 executionStats 前后对比

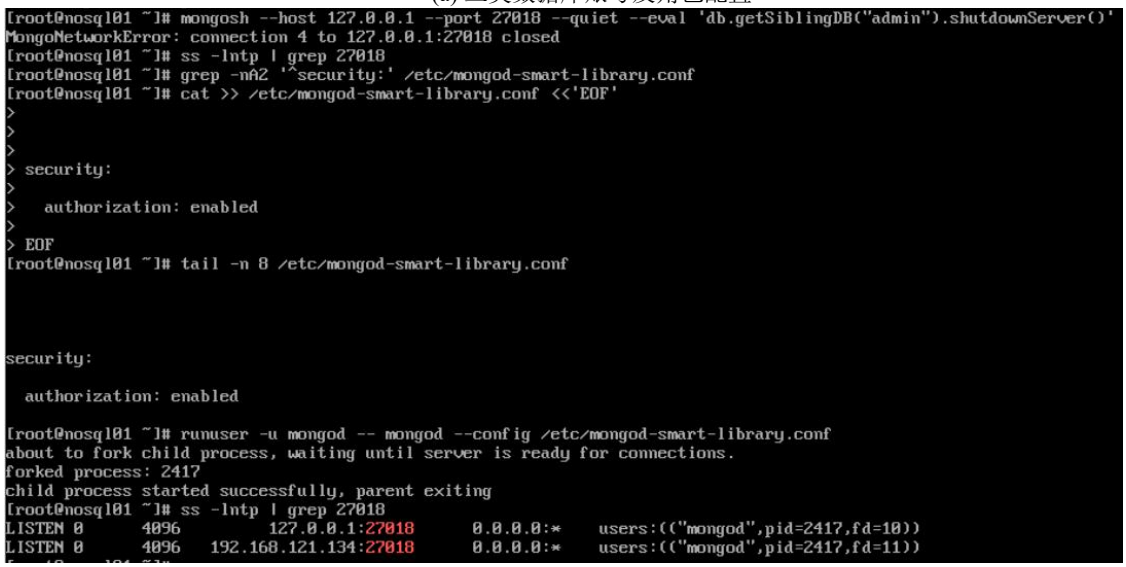
该图由索引验证脚本调用 explain() 获取执行计划，并列出业务集合已创建的索引。重点比较两类查询优化前后的扫描方式和扫描文档数：图书分类价格查询由全表扫描 50 条降至索引扫描 8 条；读者借阅历史查询由扫描 3000 条并进行排序，变为仅扫描对应读者的 20 条记录。由于数据量较小、耗时常显示为 0 ms，本项目以执行计划和扫描范围作为主要判断依据。

A.3 权限控制

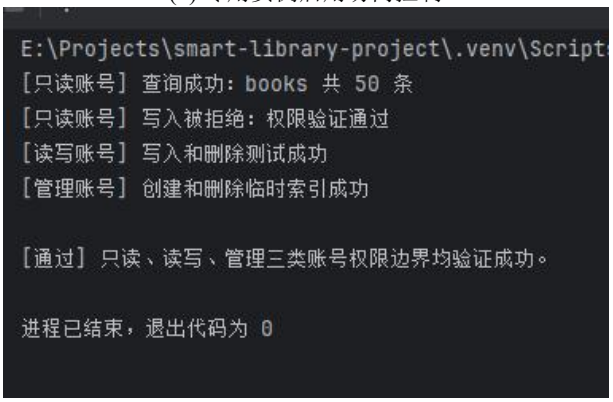
本节从账号角色配置、服务端启用 authorization 和实际操作权限测试三个层面核验访问控制，重点观察不同角色是否只能执行职责范围内的操作。



(a) 三类数据库账号及角色配置



(b) 专用实例启用访问控制



(c) 只读、读写和管理权限边界验证

图 A-6 MongoDB 用户与权限验证

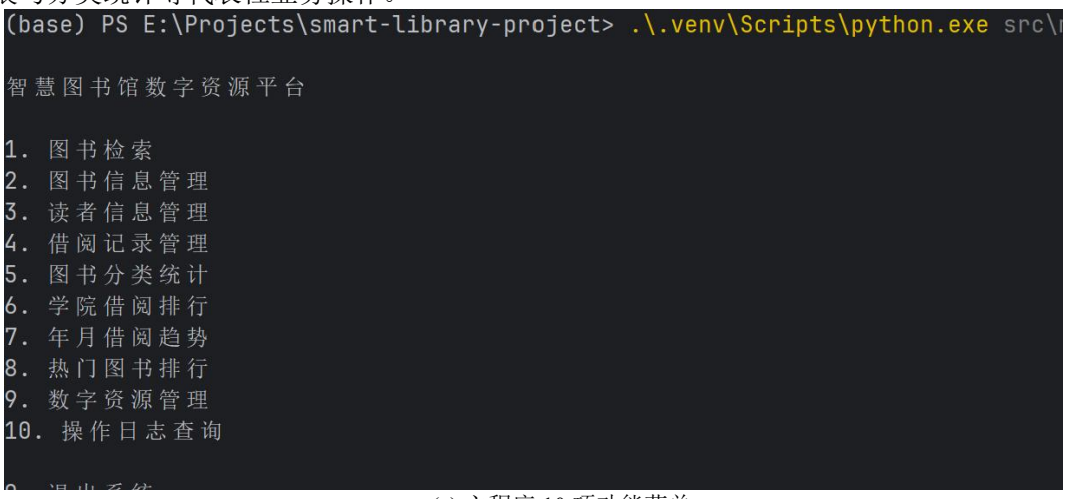
本图由用户创建脚本、实例访问控制配置和权限测试程序共同生成。截图依次展示 LibReader、LibWriter、LibAdmin 三类账号及角色配置，authorization 启用状态，以及不同账号执行操作时的结果；只读账号写入被拒绝、读写账号可完成业务操作、管理账号可执行数据库管理任务，用于验证最小权限原则和账号职责边界。

附录 B 任务四：Python 应用程序与 GridFS 文件存储

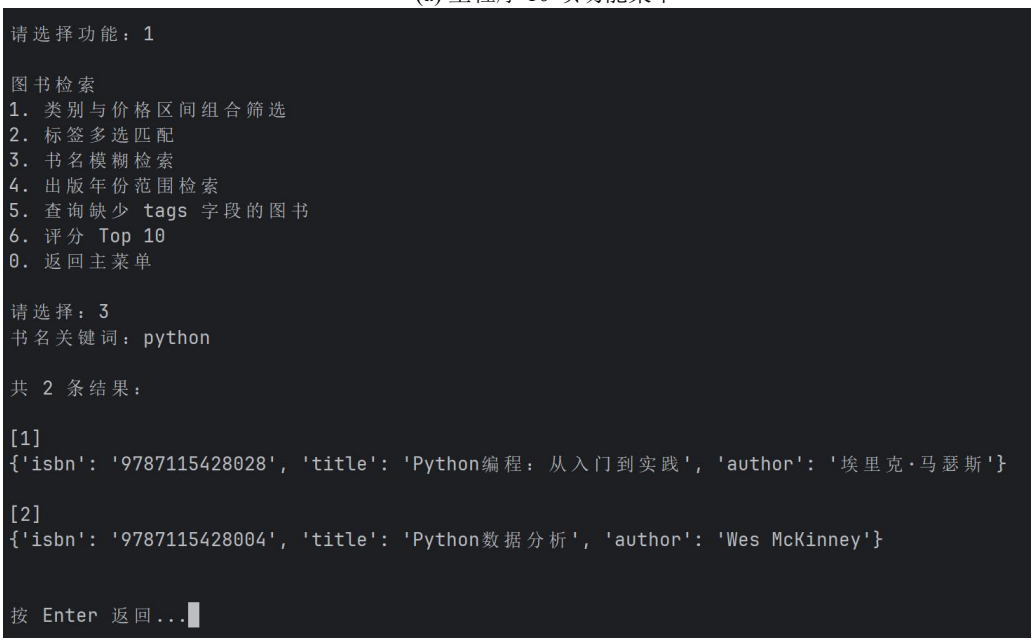
本附录对应正文第 4.5 节及第 5 章，首先展示 Python 菜单式程序、图书检索、数据导出与审计记录，然后展示 GridFS 文件上传、清单查询、业务关联和下载一致性校验，说明数据库功能如何落地为可运行的数字资源管理应用。

B.1 Python 应用程序及业务功能验证

本节展示数据库能力封装到交互式程序后的使用情况，既包括主菜单和检索，也包括数据导出、审计留痕与分类统计等代表性业务操作。



(a) 主程序 10 项功能菜单



(b) 书名模糊检索返回 2 条记录

图 B-1 Python 应用菜单与图书检索

上图为基于 PyMongo 编写的智慧图书馆主程序菜单，下图展示进入馆藏检索功能后，以书名关键词 Python 执行模糊查询并返回两条匹配记录。该图用于证明数据库操作已封装为可交互的应用功能，而非仅能通过 mongosh 手工执行命令。

```
请选择功能：2

图书信息管理
1. 按 ISBN 查询
2. 新增图书
3. 修改图书
4. 删除图书
5. 导出全部图书
0. 返回主菜单

请选择：5
已导出 50 条图书记录：E:\Projects\smart-library-project\outputs\books_export.json
```

(a) 导出 50 条图书记录

```
请选择功能：10
查看最近多少条日志？直接回车默认为 20：10

共 1 条结果：

[1]
{'operate_time': datetime.datetime(2026, 11, 3, 2, 56, 15, 438000),
 'operator': 'libWriter',
 'operate_type': '导出图书',
 'target_collection': 'books',
 'target_id': 'ALL',
 'operate_detail': '导出 50 条图书记录至 books_export.json',
 'client_ip': None}


按 Enter 返回...
```

(b) 图书导出审计日志

```
请选择功能：5

共 6 条结果：

[1]
{'book_count': 5, 'category': '历史', 'avg_price': 95.6, 'avg_rating': 4.66}

[2]
{'book_count': 5, 'category': '外语', 'avg_price': 59.0, 'avg_rating': 4.32}

[3]
{'book_count': 6, 'category': '数学', 'avg_price': 40.67, 'avg_rating': 4.43}

[4]
{'book_count': 12, 'category': '文学', 'avg_price': 47.02, 'avg_rating': 4.62}

[5]
{'book_count': 7, 'category': '经济管理', 'avg_price': 90.86, 'avg_rating': 4.3}

[6]
{'book_count': 15, 'category': '计算机', 'avg_price': 89.53, 'avg_rating': 4.53}
```

(c) 分类统计返回 6 类结果

图 B-2 Python 应用代表性功能验证

本图选取应用程序中的三项代表性操作：导出 50 条图书记录、查询图书导出对应的 audit_logs 审计记录，以及执行分类统计并返回六类结果。截图由主程序相应菜单功能运行产生，分别验证数据导出、关键操作留痕和统计结果输出，体现业务功能与审计模块之间的联动。

B.2 GridFS 电子资源存储与校验

本节按资源上传、文件清单查询、业务关联查询、下载校验和后端集合检查的顺序展开，分别验证 GridFS 文件管理功能及其底层存储完整性。

```
请选择：1

测试数字资源批量上传结果：
[1] ebook_1.pdf | ISBN: 9787115428028 | 类型: ebook | 大小: 39837 字节
    GridFS ID: 6aea9ee10a7fd996dff0eed4 | 状态: 上传成功
[2] ebook_2.pdf | ISBN: 9787121362726 | 类型: ebook | 大小: 38524 字节
    GridFS ID: 6aea9ee10a7fd996dff0eed7 | 状态: 上传成功
[3] ebook_3.pdf | ISBN: 9787115546081 | 类型: ebook | 大小: 37004 字节
    GridFS ID: 6aea9ee10a7fd996dff0eeda | 状态: 上传成功
[4] cover_1.png | ISBN: 9787115428028 | 类型: cover | 大小: 23611 字节
    GridFS ID: 6aea9ee10a7fd996dff0eedd | 状态: 上传成功
[5] cover_2.png | ISBN: 9787121362726 | 类型: cover | 大小: 26053 字节
    GridFS ID: 6aea9ee10a7fd996dff0eee0 | 状态: 上传成功
[6] cover_3.png | ISBN: 9787115546081 | 类型: cover | 大小: 15171 字节
    GridFS ID: 6aea9ee10a7fd996dff0eee3 | 状态: 上传成功

按 Enter 返回...
```

(a) 批量上传 3 个 PDF 和 3 张封面

```
请选择：2

GridFS 文件共 6 个：
[1] ebook_1.pdf | 39837 字节 | 2026-11-04 02:56:33
    ISBN: 9787115428028 | 类型: ebook | 原始文件名: ebook_1.pdf
    GridFS ID: 6aea9ee10a7fd996dff0eed4
[2] ebook_2.pdf | 38524 字节 | 2026-11-04 02:56:33
    ISBN: 9787121362726 | 类型: ebook | 原始文件名: ebook_2.pdf
    GridFS ID: 6aea9ee10a7fd996dff0eed7
[3] ebook_3.pdf | 37004 字节 | 2026-11-04 02:56:33
    ISBN: 9787115546081 | 类型: ebook | 原始文件名: ebook_3.pdf
    GridFS ID: 6aea9ee10a7fd996dff0eeda
[4] cover_1.png | 23611 字节 | 2026-11-04 02:56:33
    ISBN: 9787115428028 | 类型: cover | 原始文件名: cover_1.png
    GridFS ID: 6aea9ee10a7fd996dff0eedd
[5] cover_2.png | 26053 字节 | 2026-11-04 02:56:33
    ISBN: 9787121362726 | 类型: cover | 原始文件名: cover_2.png
    GridFS ID: 6aea9ee10a7fd996dff0eee0
[6] cover_3.png | 15171 字节 | 2026-11-04 02:56:33
    ISBN: 9787115546081 | 类型: cover | 原始文件名: cover_3.png
    GridFS ID: 6aea9ee10a7fd996dff0eee3

按 Enter 返回...
```

(b) GridFS 六个文件清单

```
请选择：4
书名关键词：Python编程

按书名查询到 1 个电子书：
[1] Python编程：从入门到实践 | 作者: 埃里克·马瑟斯 | ISBN: 9787115428028
    电子书: ebook_1.pdf | 39837 字节 | GridFS ID: 6aea9ee10a7fd996dff0eed4
```

(c) 按书名查询电子书

图 B-3 GridFS 上传 清单与业务查询

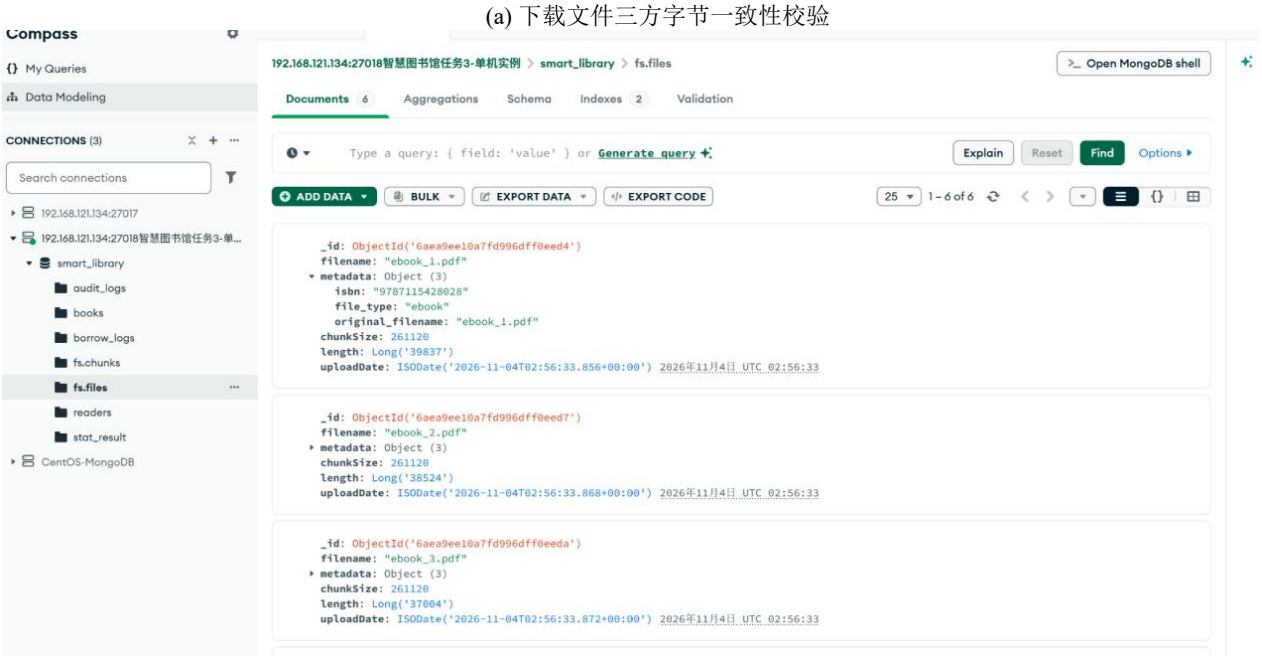
图中依次展示批量上传 3 份 PDF 电子书和 3 张封面、GridFS 文件列表，以及按书名关键词定位电子书的结果。相关操作通过项目的 GridFS 管理模块完成，文件元数据包含 ISBN、资源类型和 GridFS 文件 ID，用于验证电子资源可以上传、登记，并能通过图书业务信息进行查询关联。

```
请选择：6
电子书 GridFS ID：6aea9ee10a7fd996dff0eed4

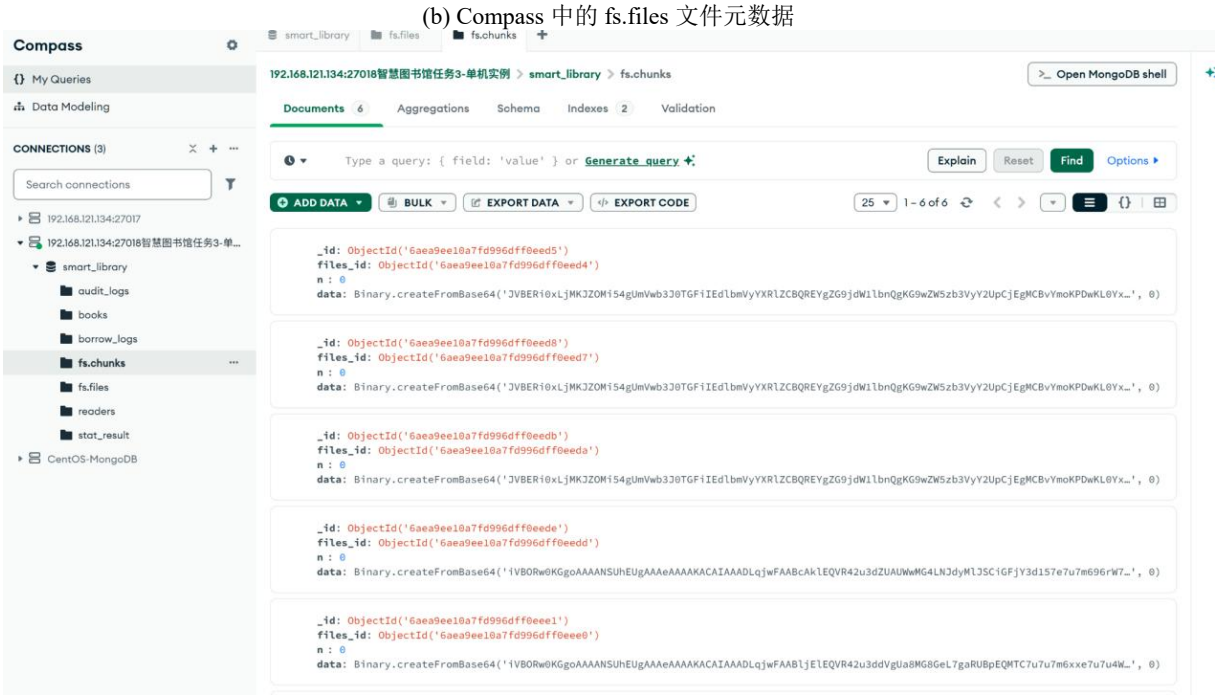
原始文件大小：39837 字节
GridFS 文件长度：39837 字节
下载文件大小：39837 字节
本地路径：E:\Projects\smart-library-project\outputs\gridfs_downloads\ebook_1.
pdf
校验结论：校验通过

按 Enter 返回...
```

(a) 下载文件三方字节一致性校验



(b) Compass 中的 fs.files 文件元数据


Compass
My Queries
Data Modeling
CONNECTIONS (3)
Search connections
192.168.121.134:27017
192.168.121.134:27018智慧图书馆任务3-单...
smart_library
audit_logs
books
borrow_logs
fs.chunks
fs.files
readers
stat_result
CentOS-MongoDB
smart_library
fs.files
fs.chunks
192.168.121.134:27018智慧图书馆任务3-单机实例 > smart_library > fs.chunks
Open MongoDB shell
Documents 6
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1-6 of 6
_id: ObjectId('6aea9ee10a7fd996dff0eed5')
files_id: ObjectId('6aea9ee10a7fd996dff0eed4')
n: 0
data: Binary.createFromBase64('JVBERi0xLjMKJZOMi54gUmVwb3J0TGFiIEdlbmVyYXRlZCBQREYgZG9jdWllbnQgKG9wZW5zb3VyY2UpCjEgMCBvYmoKPDwKL0Yx…', 0)
_id: ObjectId('6aea9ee10a7fd996dff0eed8')
files_id: ObjectId('6aea9ee10a7fd996dff0eed7')
n: 0
data: Binary.createFromBase64('JVBERi0xLjMKJZOMi54gUmVwb3J0TGFiIEdlbmVyYXRlZCBQREYgZG9jdWllbnQgKG9wZW5zb3VyY2UpCjEgMCBvYmoKPDwKL0Yx…', 0)
_id: ObjectId('6aea9ee10a7fd996dff0eedb')
files_id: ObjectId('6aea9ee10a7fd996dff0eeda')
n: 0
data: Binary.createFromBase64('JVBERi0xLjMKJZOMi54gUmVwb3J0TGFiIEdlbmVyYXRlZCBQREYgZG9jdWllbnQgKG9wZW5zb3VyY2UpCjEgMCBvYmoKPDwKL0Yx…', 0)
_id: ObjectId('6aea9ee10a7fd996dff0eede')
files_id: ObjectId('6aea9ee10a7fd996dff0eedd')
n: 0
data: Binary.createFromBase64('iVBORw0KGgoAAAANSUhEUgAAAeAAAAKACAIAAADLqjwFAABcAklEQVR42u3dZUAUWwMwMG4LNJdyb2…', 0)
_id: ObjectId('6aea9ee10a7fd996dff0eee1')
files_id: ObjectId('6aea9ee10a7fd996dff0eee0')
n: 0
data: Binary.createFromBase64('iVBORw0KGgoAAAANSUhEUgAAAeAAAAKACAIAAADLqjwFAABljElEQVR42u3ddVgUa8MG8GGwICYICYICY

数据集合和 fs.chunks 分块集合。校验比较源文件、GridFS 存储内容与下载文件的字节长度及一致性结果；后两幅截图用于从数据库端核对文件元数据和二进制分块确已写入，形成业务层与存储层相互印证的证据。

附录 C 任务五：副本集部署与高可用验证

本附录对应正文第 6.2 节，按副本集部署、数据复制、读取偏好、故障转移、认证和成员管理的实际操作顺序排列。正文重点呈现三节点最终状态、故障转移结论和访问控制结果；本附录补充关键命令执行及节点变化过程，帮助追溯各项结论的形成依据。

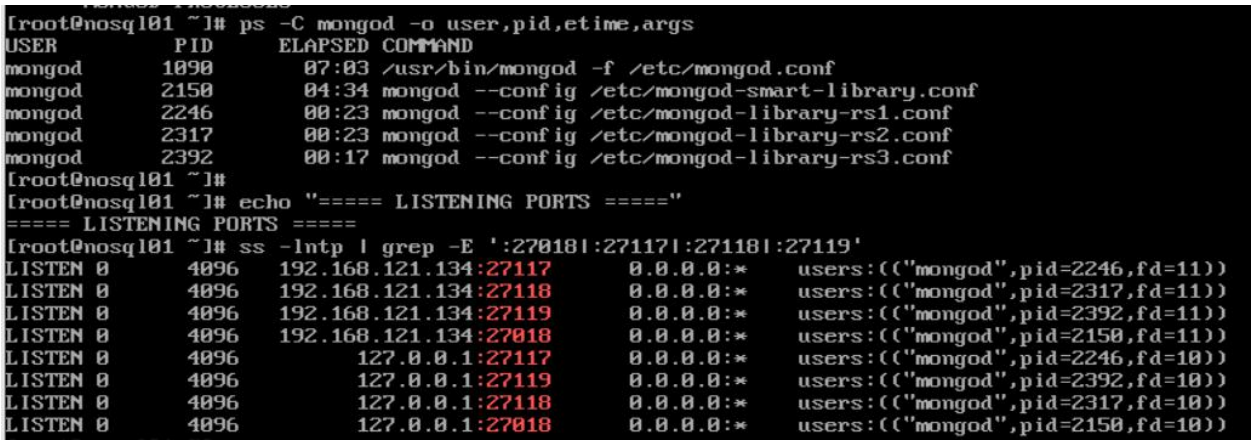


图 C-1 三节点实例启动及端口监听

该截图通过进程列表和端口监听检查获得，分别核对 27117、27118、27119 三个 mongod 实例是否启动并监听预定端口。它是副本集初始化前的环境检查证据，说明三个节点使用独立实例运行，为后续建立 libraryRS 和验证节点角色提供基础。

27117、27118、27119 均由独立 mongod 进程监听，为副本集初始化提供运行基础。

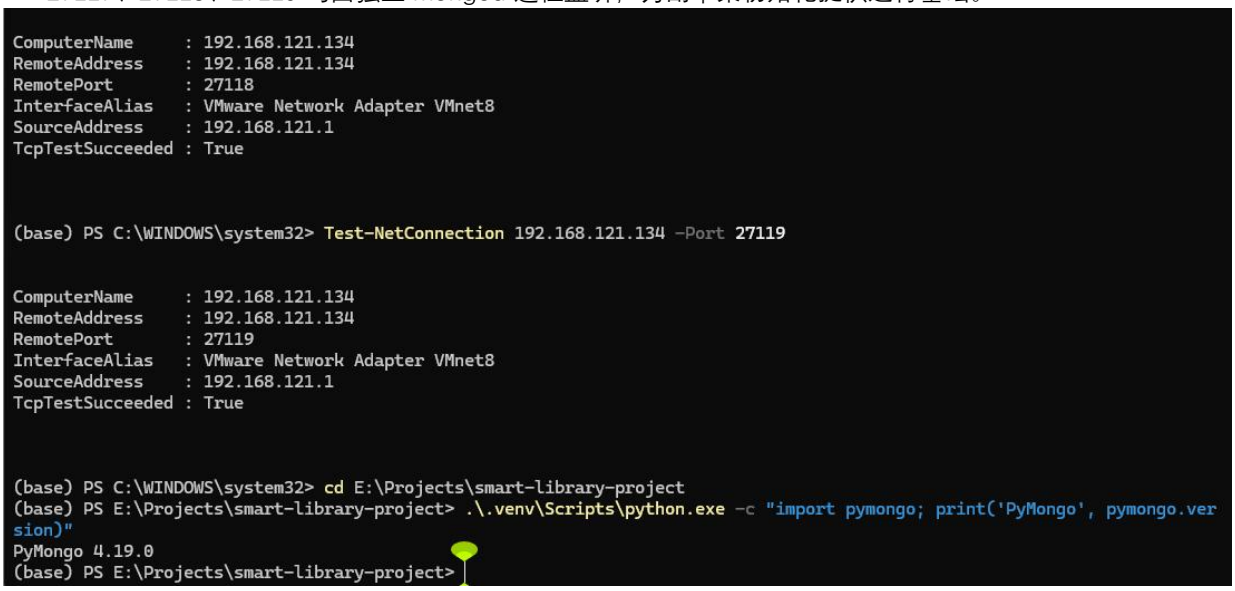


图 C-2 Windows 客户端副本集连通性验证

本图由 Windows 端执行网络连通性测试并检查 PyMongo 环境所得，验证客户端能够访问虚拟机中的副本集节点端口，且 Python 驱动可正常导入。该步骤用于排除客户端网络或运行环境问题，确保后续故障转移探针能够从应用端观察副本集状态。

Windows 端能够访问虚拟机地址并运行 PyMongo 环境，为应用层探针提供客户端条件。



图 C-3 PRIMARY 写入复制测试文档

在 libraryRS 初始化完成后，实验连接当前 PRIMARY，向 replica_test.replication_probe 集合插入一条带有任务标识和写入节点信息的测试文档。返回 acknowledged=true 表示写入请求得到确认；该文档随后作为两个 SECONDARY 读取验证的共同样本。

主节点向 replica_test.replication_probe 写入测试文档并返回 acknowledged=true。

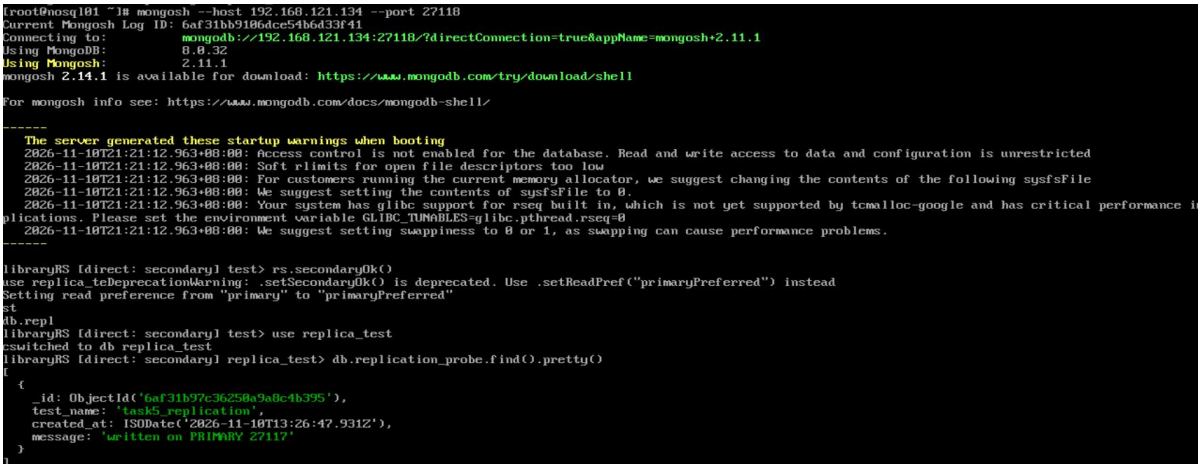


图 C-4 SECONDARY 27118 读取同步数据

该图是在连接 27118 并设置 secondary 读取偏好后查询 replication_probe 得到的结果。查询能够读到 PRIMARY 写入的同一条测试文档，说明该节点已接收并应用复制数据，可作为副本集数据同步的过程证据。

27118 以 secondary 读取偏好获取主节点写入的测试记录。

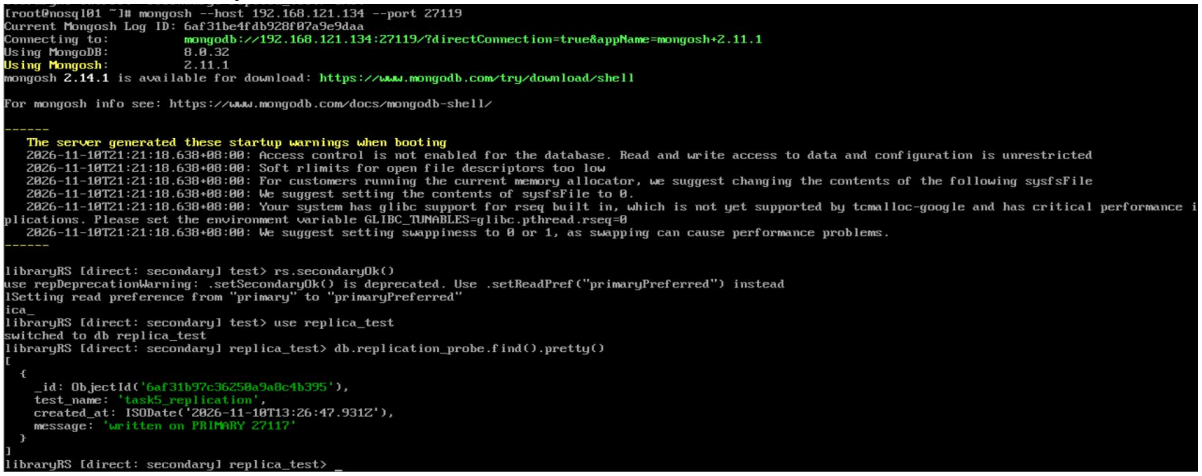


图 C-5 SECONDARY 27119 读取同步数据

该截图采用与上一图相同的验证方法，在 27119 上读取同一测试集合。两个 SECONDARY 均能查到相同记录，补足了单个从节点验证的不足，说明本次测试数据已同步到两个从节点。

27119 读取到同一测试记录，形成两个 SECONDARY 的同步证据。



图 C-6 primary 读取偏好验证

该图通过客户端设置 primary 读取偏好后执行查询，并检查实际读取节点与返回记录。结果显示读取由 PRIMARY 提供，用于验证 primary 模式将查询固定到主节点，适用于对数据新鲜度要求较高的业务读取场景。

查询结果显示读取由 PRIMARY 提供，验证 primary 模式配置有效。

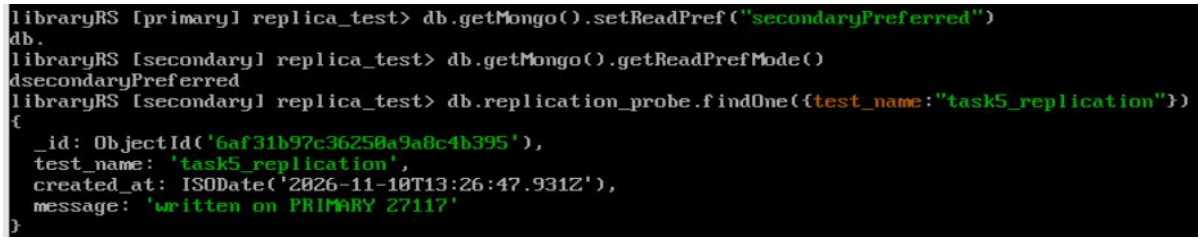


图 C-7 secondaryPreferred 读取偏好验证

该图通过客户端设置 secondaryPreferred 读取偏好后执行查询获得。结果显示当前查询优先由 SECONDARY 提供，验证了从节点优先读取策略；该模式可用于能够容忍短暂复制延迟的馆藏浏览或统计读取，但不应替代库存、权限等需要较新数据的关键校验。

查询结果显示 secondaryPreferred 模式能够优先使用 SECONDARY。

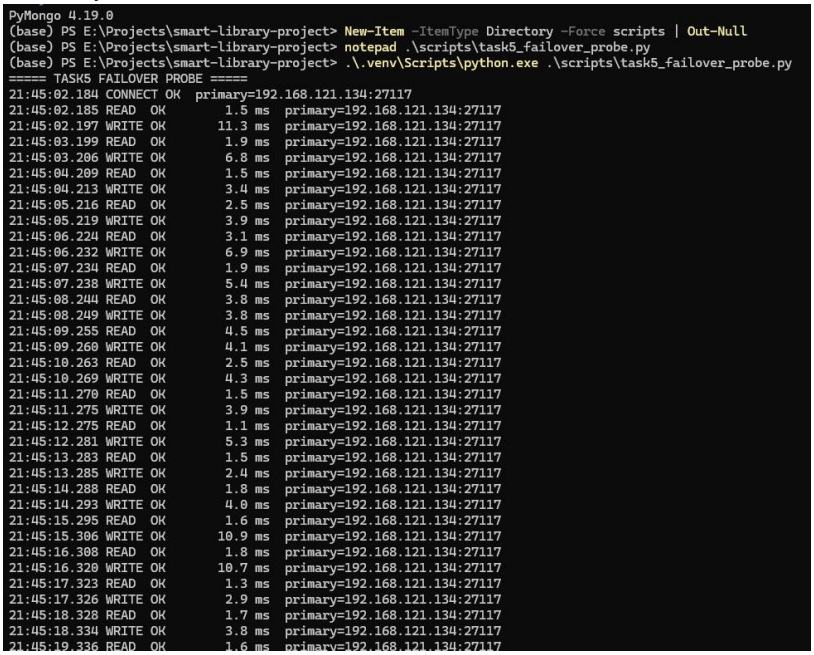


图 C-8 故障转移前 Python 连续读写基线

该截图来自故障演练前运行的 Python/PyMongo 连续读写探针。日志持续输出 READ OK 和 WRITE

OK，并识别 27117 为 PRIMARY，作为故障发生前的正常运行基线；后续需与主节点停止后的选举和客户端恢复记录结合判断。

探针在正常拓扑下连续输出 READ OK 和 WRITE OK，并识别 27117 为 PRIMARY。

```
[root@nosql01 ~]# runuser -u mongod -- mongod --config /etc/mongod-library-rs1.conf
about to fork child process, waiting until server is ready for connections.
forked process: 2959
child process started successfully, parent exiting
[root@nosql01 ~]# sleep 5
[root@nosql01 ~]# mongosh --host 192.168.121.134 --port 27118 --quiet --eval '
> printjson(rs.status().members.map(m => ({
>   name: m.name,
>   state: m.stateStr,
>   health: m.health
> })))'
[
  {
    name: '192.168.121.134:27117',
    state: 'SECONDARY',
    health: 1
  },
  {
    name: '192.168.121.134:27118',
    state: 'PRIMARY',
    health: 1
  },
  {
    name: '192.168.121.134:27119',
    state: 'SECONDARY',
    health: 1
  }
]
```

图 C-9 原 PRIMARY 重启后的角色恢复

在故障转移验证后重新启动原 27117 实例，再通过副本集状态命令检查成员角色和 health 字段。截图显示 27117 重新加入并以 SECONDARY 身份运行，其他节点角色保持正常，证明原节点恢复后能够重新追赶并加入副本集，而不是直接抢占主节点。

重启 27117 后，节点重新加入副本集并恢复为健康成员。

```
libraryRS [direct: primary] admin> db.createUser({
|   user: "libraryAdmin",
|   pwd: passwordPrompt(),
|   roles: [
|     { role: "root", db: "admin" }
|   ]
| })
Enter password
**************{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1794319736, i: 3 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1794319736, i: 3 })
}
```

图 C-10 管理账号创建与认证配置

该图记录在 PRIMARY 上创建 libraryAdmin 管理账号，并为三个副本集节点配置共享 keyfile 与 authorization 的过程。账号创建结果和认证配置共同用于验证副本集不仅完成节点复制，也启用了节点间内部认证及客户端访问控制；正文中的权限拒绝与管理员认证结果进一步构成验收依据。

创建 libraryAdmin，并在三个副本集节点启用共享 keyfile 和 authorization。

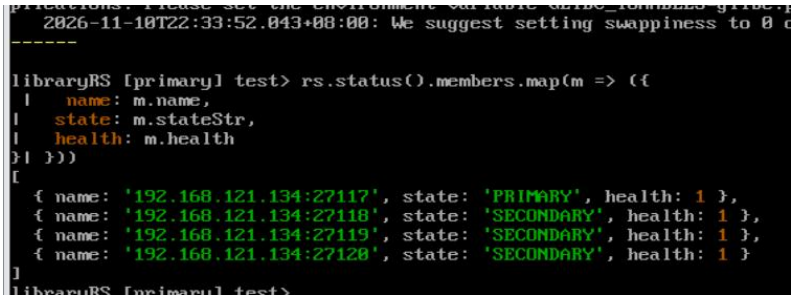


图 C-11 临时节点 27120 加入副本集

通过 rs.add() 将临时节点 192.168.121.134:27120 加入 libraryRS，再查询副本集成员状态得到该截图。结果显示新增节点成为 SECONDARY，且四个成员 health 均为 1，用于验证副本集可以在运行过程中扩展成员。

27120 加入后成为 SECONDARY，四个成员均 health=1。

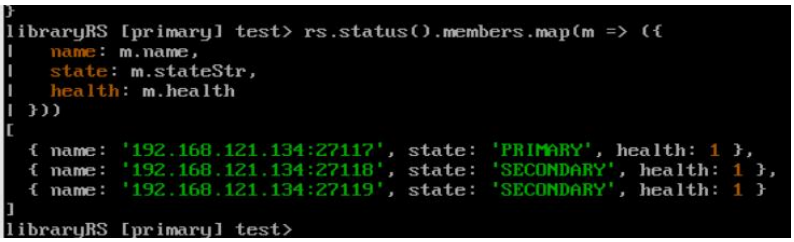


图 C-12 临时节点移除后的三节点状态

完成临时扩展验证后，通过 rs.remove() 移除 27120，并再次检查成员列表。截图显示副本集恢复为三个正式节点，角色为 1 PRIMARY、2 SECONDARY，且成员健康状态正常，证明节点加入和移除操作均已完成并恢复到预定拓扑。

移除 27120 后，副本集恢复为三个正式成员，1 PRIMARY + 2 SECONDARY。

附录 D 任务六：分片集群部署与扩展验证

本附录对应正文第 6.3 节，按 Config Server 初始化、Shard 初始化、集群状态检查、数据导出、分片键配置、Chunk 检查、数据恢复和认证验收的顺序排列。正文已展示分片键论证、最终文档分布及查询路由等核心结果，因此本附录侧重保留集群搭建和验收过程证据，避免重复堆放相同结论截图。

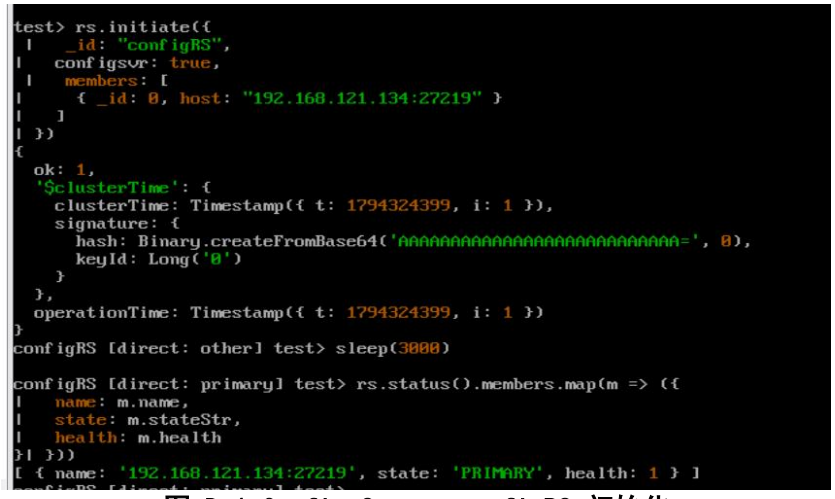
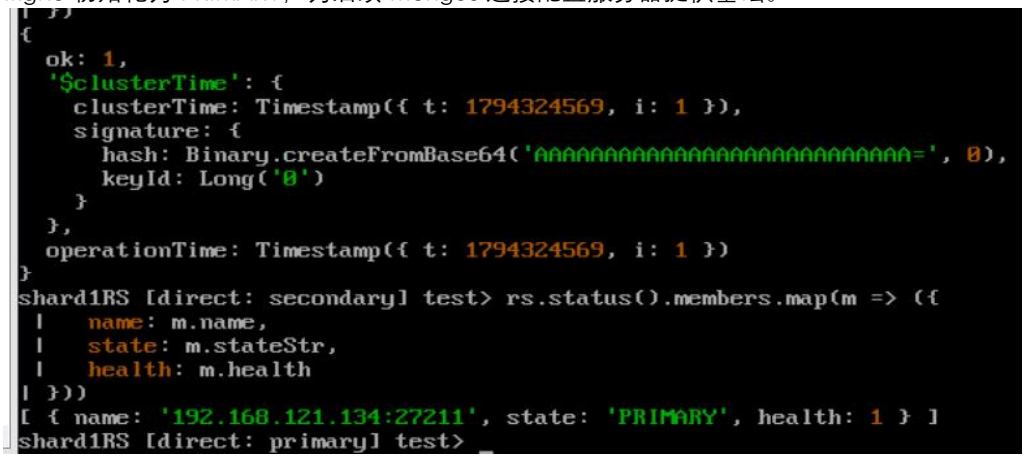


图 D-1 Config Server configRS 初始化

该截图来自在配置服务器实例上执行 rs.initiate() 并随后检查 rs.status() 的过程。configRS 成功形成 PRIMARY，说明配置服务器副本集已完成初始化，可为 mongos 提供集群元数据服务，是分片集群启动的基础步骤。

configRS 初始化为 PRIMARY，为后续 mongos 连接配置服务器提供基础。



(a) shard1RS



(b) shard2RS

图 D-2 两个 Shard 副本集初始化状态

图中分别展示 shard1RS 和 shard2RS 的初始化命令及状态检查结果。两个 Shard 均以单成员副本集形式形成 PRIMARY，为后续注册到集群、接收分片数据提供运行基础；该单成员设计符合本次课程虚拟机资源约束，不代表生产环境的冗余部署。

shard1RS 和 shard2RS 分别初始化并形成各自的单成员 PRIMARY。



图 D-3 分片集群初始状态与 Balancer 状态

该截图通过 mongos 执行集群状态和分片分布相关命令获得，用于核对 mongos 已能连接配置服务器、Shard 已注册，以及 Balancer 当前状态。它记录的是数据恢复前的集群初始状态，便于与后续数据导入和最终分布结果区分。

初始状态截图用于核对 mongos、Shard 列表和均衡器状态。

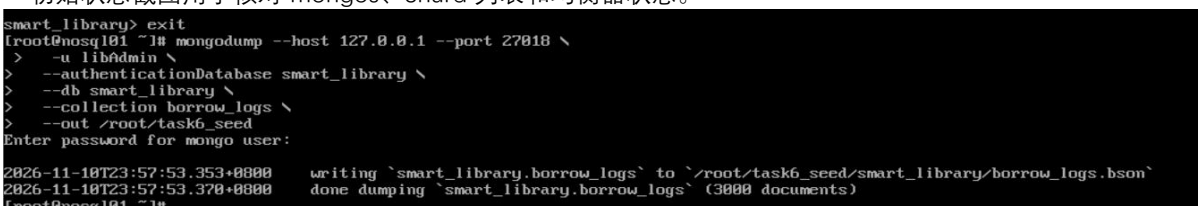


图 D-4 任务三 borrow_logs 导出结果

该图展示从任务三单机环境导出 smart_library.borrow_logs 的命令和执行结果。导出的数据文件作为任务六恢复数据源，使分片实验沿用前期已验证的 3000 条借阅流水，而不是另行构造一份测试数据。

从任务三单机数据库导出 smart_library.borrow_logs，作为分片集群恢复的数据源。

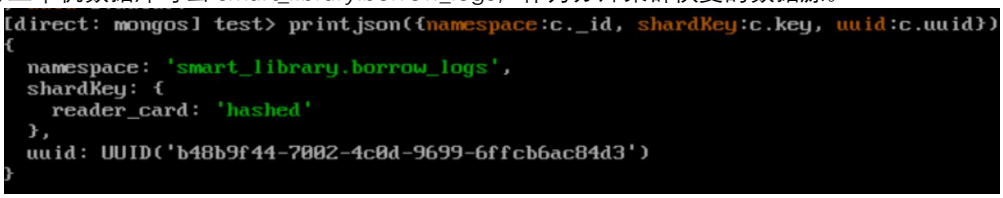


图 D-5 borrow_logs 分片键配置

该截图由 mongos 连接后查询集合分片配置得到，显示 smart_library.borrow_logs 使用 {reader_card: "hashed"} 作为分片键。配置依据来自正文对 reader_card、book_isbn 基数及查询模式的比较；选择读者借阅证号主要考虑读者历史记录的定向查询需求和哈希键分散写入的作用。

在 smart_library.borrow_logs 上启用 {reader_card:"hashed"} 分片键。



图 D-6 初始 Chunk 分布

分片键启用后，通过集群分布信息命令查看 Chunk 归属，截图显示 shard1RS 与 shard2RS 各拥有一个初始 Chunk。该结果说明分片元数据已建立；它只反映初始 Chunk 分配，不能单独代表 3000 条业务文档已经均匀分布，实际文档数量应以正文最终统计为准。

配置完成后两个 Shard 各拥有 1 个初始 Chunk。

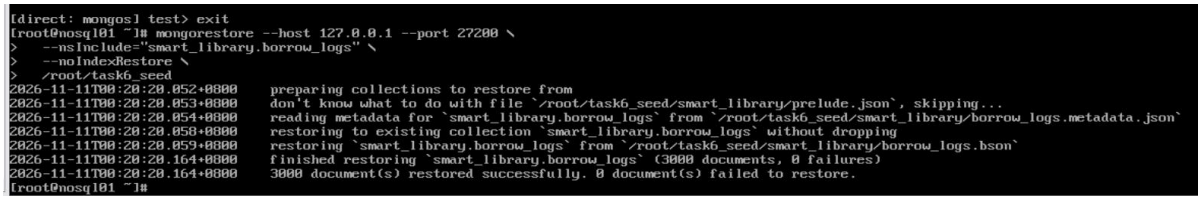


图 D-7 通过 mongos 恢复 3000 条借阅流水

该图记录通过 mongos 端口 27200 执行 mongorestore 的恢复过程。命令输出显示 3000 documents restored successfully、0 failures，说明从单机环境导出的借阅流水已成功写入分片集群，后续可据此检查两个 Shard 的实际数据分布和查询路由。

mongorestore 显示 3000 documents restored successfully，0 failures。

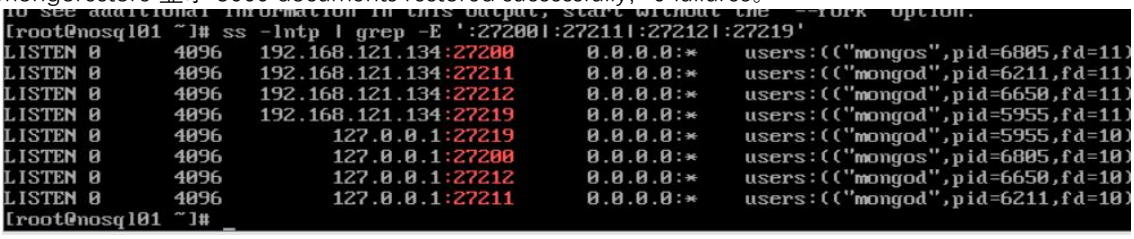


图 D-8 认证配置重启后的组件监听状态

该截图通过端口监听命令检查启用认证并重启后的集群组件，核对 27200、27211、27212 和 27219 均已恢复监听。该检查用于确认 mongos、两个 Shard 和 Config Server 均重新启动成功；结合正文对旧 PID 占用 27212 的排查记录，可说明重启后端口冲突已解决。

认证重启后 27200、27211、27212 和 27219 均恢复监听。



（a）未认证访问被拒绝



（b）管理员认证成功并正常访问两个 Shard

图 D-9 分片集群访问控制验证

图中包含未认证访问 listShards 被拒绝，以及使用 shardingAdmin 认证后能够查看两个 Shard 的结果。截图由 mongos 端执行相同管理命令、对比认证前后响应得到，用于证明集群访问控制已生效，且授权管理员能够正常读取分片集群元数据。

未认证 listShards 被拒绝；shardingAdmin 认证后能够访问两个 Shard。

附录 E 任务七：图形化验收与结果核对

本附录对应正文第 6.3.5 节及项目总结中的独立核验工作，分别保留 Compass 对借阅流水总量的检查、Compass 借阅状态聚合结果，以及 mongosh 的同口径聚合输出。通过图形化界面与命令行两种方式交叉核对，验证数据总量和聚合统计结果的一致性。

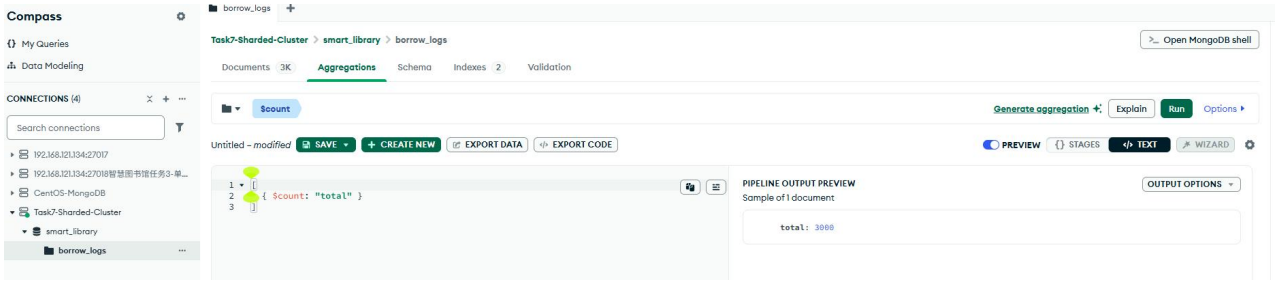


图 E-1 Compass 核对 borrow_logs 数据量为 3000 条

该截图由 MongoDB Compass 连接分片集群后查看 smart_library.borrow_logs 集合得到，用图形化界面核对集合总文档数为 3000。它与 mongosh 端的 countDocuments 结果形成独立交叉检查，用于确认恢复数据没有因分片部署或认证重启而丢失。

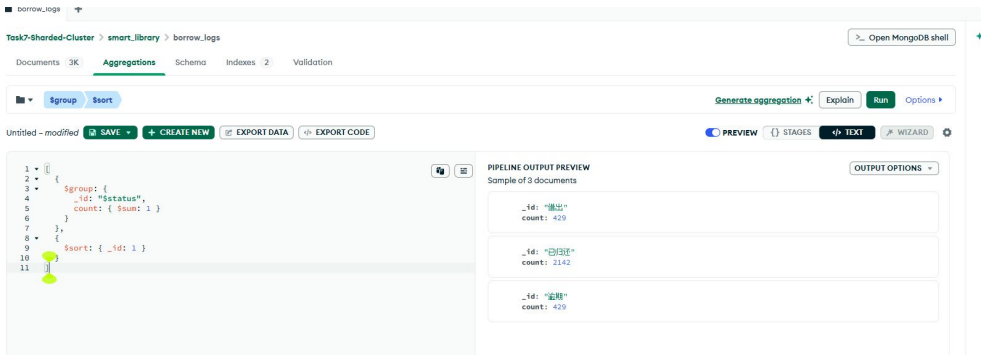


图 E-2 Compass 借阅状态聚合结果

该图展示在 Compass 聚合管道界面中对 borrow_logs 按借阅状态分组统计的结果。通过图形化方式查看各状态对应的记录数量，便于检查聚合分组字段、统计结果及其输出结构，并为命令行结果对照提供参照。

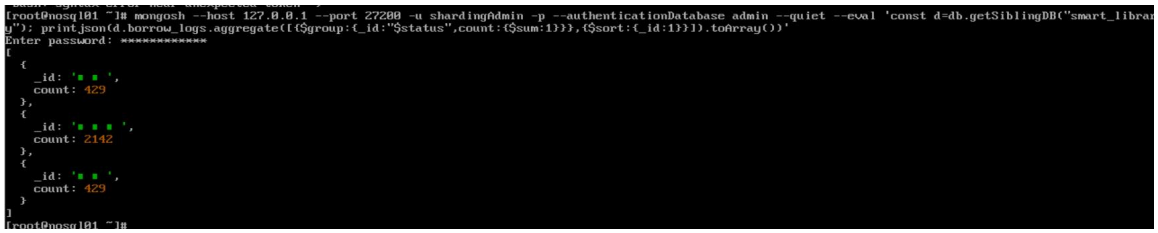


图 E-3 mongosh 借阅状态聚合结果

该截图由 mongosh 执行与 Compass 相同口径的借阅状态聚合操作得到。将命令行输出与图 D-2 的 Compass 结果逐项核对，可验证图形化操作与数据库实际聚合结果一致；两种界面互为校验，减少单一客户端展示造成的误判。